Password Decryption

Problem Statement

Given an encrypted string and a list of indices indicating the sequence to read characters, reconstruct the original password.

Variants

- Multiple records: decrypt a batch of strings with corresponding index lists.

- File

How to Leverage the Solution

Normalize/parsing once for indices, then apply the same O(n) indexing to build the result.

Java Solution

Below is a concise reference implementation. Full comments omitted for brevity.

```
import java.util.*;
import java.io.*;
import java.nio.file.*;

public final class PasswordDecryptor {
 public static String decryptByIndices(String cipher, List<Integer> indices) {
  if (cipher == null || indices == null) return "";
  StringBuilder builder = new StringBuilder(indices.size());
  for (int index : indices) {
   if (index < 0 || index >= cipher.length()) {
    throw new IllegalArgumentException("Index out of bounds: " + index);
   }
   builder.append(cipher.charAt(index));
  }
  return builder.toString();
 }

 public static List<String> decryptBatch(List<String> ciphers, List<List<Integer>> indicesLists) {
  if (ciphers.size() != indicesLists.size()) {
   throw new IllegalArgumentException("Mismatched batch sizes");
  }
  List<String> results = new ArrayList<>(ciphers.size());
  for (int i = 0; i < ciphers.size(); i++) {
   results.add(decryptByIndices(ciphers.get(i), indicesLists.get(i)));
  }
  return results;
 }

 public static List<String> decryptFromFile(Path file) throws IOException {
  List<String> decrypted = new ArrayList<>();
  try (BufferedReader reader = Files.newBufferedReader(file)) {
   String cipherLine;
   while ((cipherLine = reader.readLine()) != null) {
    String indicesLine = reader.readLine();
    if (indicesLine == null) break;
    String cipher = parseAfterColon(cipherLine);
    List<Integer> indices = parseIndices(parseAfterColon(indicesLine));
    decrypted.add(decryptByIndices(cipher, indices));
   }
```

```
  }
  return decrypted;
 }

 private static String parseAfterColon(String line) {
  int idx = line.indexOf(':');
  return idx >= 0 ? line.substring(idx + 1).trim() : line.trim();
 }

 private static List<Integer> parseIndices(String s) {
  if (s.isEmpty()) return Collections.emptyList();
  String[] parts = s.split("[,\\s]+");
  List<Integer> result = new ArrayList<>(parts.length);
  for (String p : parts) {
   if (p.isEmpty()) continue;
   result.add(Integer.parseInt(p));
  }
  return result;
 }
}
```